

EpiSphere AI - Complete Platform Documentation

Table of Contents

- 1. [Platform Overview](#)
 - 2. [Architecture](#)
 - 3. [Features](#)
 - 4. [User Roles and Permissions](#)
 - 5. [API Documentation](#)
 - 6. [Database Schema](#)
 - 7. [Machine Learning Components](#)
 - 8. [Frontend Structure](#)
 - 9. [Setup and Installation](#)
 - 10. [Configuration](#)
 - 11. [Security](#)
 - 12. [Development Guide](#)
 - 13. [Deployment](#)
 - 14. [Troubleshooting](#)
-

Platform Overview

EpiSphere AI is a production-ready, AI-powered global disease surveillance and outbreak intelligence platform designed for Ministries of Health, epidemiologists, and public health organizations worldwide.

Purpose

The platform provides:

- **Real-time global disease surveillance** with interactive dashboards
- **AI-powered outbreak detection** using multiple machine learning algorithms
- **Forecasting capabilities** for short-term disease trends
- **Automated alerting system** for early outbreak signals
- **Data management** with CSV/Excel upload and API integration
- **Reporting tools** for epidemiological bulletins and analysis

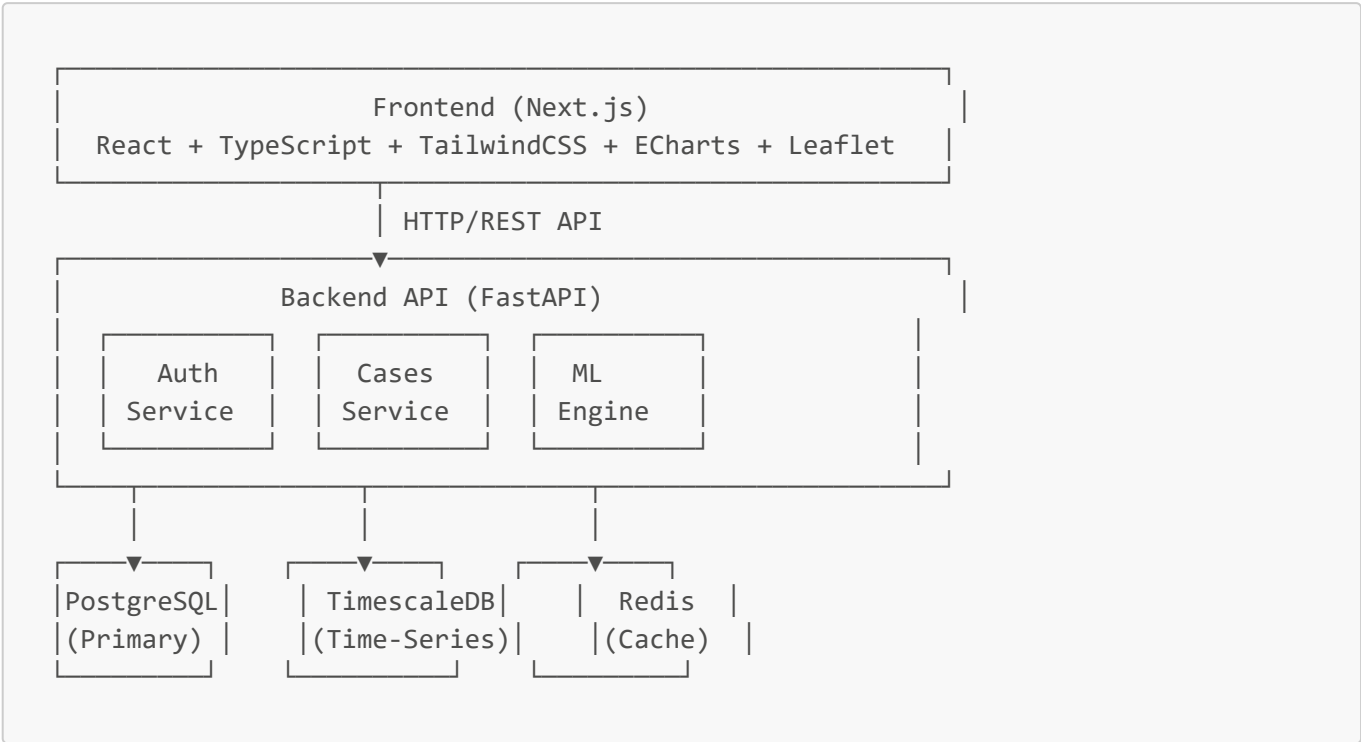
Key Capabilities

- Monitor disease cases across countries in real-time
 - Detect early outbreak signals using advanced ML algorithms
 - Generate short-term forecasts (7-30 days) with confidence intervals
 - Manage and track alerts with severity classification
 - Upload and validate case data in bulk
 - Generate automated epidemiological reports
 - Role-based access control for different user types
-

Architecture

System Architecture

EpiSphere AI follows a modern microservices architecture with clear separation of concerns:



Technology Stack

Backend

- **Framework:** FastAPI 0.104.1
- **Language:** Python 3.10+
- **ORM:** SQLAlchemy 2.0.23
- **Migrations:** Alembic 1.12.1
- **Database:** PostgreSQL 15 with TimescaleDB extension
- **Cache:** Redis 7
- **Authentication:** JWT (python-jose)
- **Validation:** Pydantic 2.5.0

Frontend

- **Framework:** Next.js 14.0.4
- **Language:** TypeScript 5.3.3
- **UI Library:** React 18.2.0
- **Styling:** TailwindCSS 3.3.6
- **Charts:** Apache ECharts 5.4.3
- **Maps:** Leaflet 1.9.4 + React-Leaflet
- **Forms:** React Hook Form + Zod validation
- **HTTP Client:** Axios 1.6.2

Machine Learning

- **General ML:** scikit-learn 1.3.2
- **Time Series:** statsmodels 0.14.0
- **Forecasting:** Prophet 1.1.5
- **Deep Learning:** PyTorch 2.1.1
- **Utilities:** NumPy 1.26.2, Pandas 2.1.3

DevOps

- **Containerization:** Docker + Docker Compose
- **Database Migrations:** Alembic
- **Environment Management:** python-dotenv

Project Structure



```

├── dashboard_service.py
├── data_upload.py
├── forecast_service.py
├── outbreak_detection_service.py
├── report_service.py
├── main.py                # FastAPI application entry point
├── alembic/               # Database migrations
├── env.py
├── versions/
├── scripts/              # Utility scripts
├──   ├── init_db.py       # Database initialization
├──   └── run_outbreak_detection.py
├── requirements.txt       # Python dependencies
├── requirements-dev.txt   # Development dependencies
├── Dockerfile            # Backend container definition

├── frontend/             # Next.js frontend application
├──   ├── app/             # Next.js app directory (pages)
├──   ├──   ├── page.tsx   # Landing page
├──   ├──   ├── layout.tsx # Root layout
├──   ├──   ├── globals.css # Global styles
├──   ├──   ├── auth/      # Authentication pages
├──   ├──   ├──   ├── login/
├──   ├──   ├──   └── register/
├──   ├──   ├── dashboard/ # Dashboard pages
├──   ├──   ├──   ├── global/
├──   ├──   ├──   └── country/[id]/
├──   ├──   ├── alerts/    # Alerts page
├──   ├──   └── upload/    # Data upload page
├──   ├── components/     # React components
├──   ├──   ├── Navbar.tsx
├──   ├──   └── dashboard/
├──   ├──       ├── StatsCards.tsx
├──   ├──       ├── TimeSeriesChart.tsx
├──   ├──       └── GlobalMap.tsx
├──   ├── lib/            # Utilities and API clients
├──   ├──   └── api.ts     # API client with authentication
├──   ├── public/         # Static assets
├──   ├── package.json
├──   ├── tsconfig.json
├──   ├── tailwind.config.js
├──   └── Dockerfile      # Frontend container definition

├── docker-compose.yml    # Docker orchestration
├── README.md             # Main readme
├── SETUP.md              # Setup instructions
├── QUICK_START.md        # Quick start guide
├── START_SERVERS.md      # Server startup guide
├── PROJECT_SUMMARY.md    # Project summary
└── DOCUMENTATION.md     # This file

```

Features

1. Global Surveillance Dashboard

Purpose: Real-time monitoring of disease cases worldwide

Features:

- Interactive world map visualization showing case distribution
- Daily and cumulative case counts
- Incidence rate per 100,000 population
- Filters by:
 - Disease type
 - Date range
 - Geographic region
- Top countries ranking by case count
- Time-series visualizations

Access: </dashboard/global>

2. Country-Level Dashboard

Purpose: Detailed analysis for specific countries

Features:

- Daily cases time-series chart
- 7-day moving average trend
- Key metrics:
 - Case Fatality Rate (CFR)
 - Incidence per 100,000
 - Growth rate
 - Total cases
 - Total deaths
- Recent data table
- Subnational breakdown (when available)

Access: /dashboard/country/{country_id}

3. AI Outbreak Detection

Purpose: Early detection of potential disease outbreaks using machine learning

Detection Algorithms:

1. Baseline Statistical Threshold

- Compares current cases to historical mean + standard deviation
- Simple but effective for detecting sudden spikes

2. Isolation Forest

- Unsupervised anomaly detection
- Identifies outliers in case patterns
- Effective for non-linear patterns

3. **CUSUM (Cumulative Sum) Change Detection**

- Detects gradual changes in case trends
- Sensitive to sustained increases

4. **Seasonal Anomaly Detection**

- Accounts for seasonal patterns
- Identifies deviations from expected seasonal trends

5. **LSTM Residual Anomalies** (Future implementation)

- Deep learning approach
- Detects anomalies in LSTM model residuals

Alert Severity Levels:

- **Low:** Minor deviation from baseline
- **Moderate:** Significant increase requiring attention
- **High:** Major outbreak signal requiring immediate action

Output:

- Probability score (0-1)
- Severity classification
- Human-readable explanation
- Detection method used

Access: Automated (runs on case data upload) or via </api/v1/alerts>

4. Forecasting Engine

Purpose: Predict future disease trends for planning and resource allocation

Forecasting Models:

1. **ARIMA (AutoRegressive Integrated Moving Average)**

- Statistical time-series model
- Good for short-term forecasts
- Handles trends and seasonality

2. **Prophet**

- Facebook's forecasting tool
- Excellent for seasonal patterns
- Handles holidays and events

3. **LSTM (Long Short-Term Memory)**

- Deep learning neural network
- Can capture complex patterns
- Requires more data

Features:

- Automatic model selection based on validation performance
- Forecast horizon: 7-30 days
- Confidence intervals (upper/lower bounds)
- Model performance metrics
- Historical forecast accuracy tracking

Access: </api/v1/forecast/generate>

5. Alerts System

Purpose: Centralized management of outbreak alerts

Features:

- Alert dashboard with filtering:
 - By country
 - By disease
 - By severity (Low/Moderate/High)
 - By status (Triggered/Investigating/Resolved/False Positive)
- Alert lifecycle management:
 - Triggered → Investigating → Resolved/False Positive
- Visual severity indicators
- Alert details with:
 - Detection method
 - Probability score
 - Explanation
 - Affected location and disease
- Email notifications (architecture ready)

Access: </alerts> or </api/v1/alerts>

6. Data Upload & Integration

Purpose: Bulk import of case data from various sources

Features:

- **File Upload:**
 - Supports CSV and Excel (.xlsx, .xls)
 - Column mapping interface
 - Data validation
 - Error reporting
- **API Integration:**
 - RESTful endpoints for programmatic data submission

- Batch upload support
- **Data Quality Checks:**
 - Required field validation
 - Date range validation
 - Duplicate detection
 - Outlier detection
- **Processing:**
 - Automatic outbreak detection on upload
 - Statistics calculation
 - Dashboard data refresh

Access: `/upload` or `/api/v1/cases/upload`

7. Reporting Module

Purpose: Generate epidemiological reports and bulletins

Report Types:

- **Weekly Bulletin:** Summary of cases for the past week
- **Monthly Report:** Comprehensive monthly analysis
- **Outbreak Report:** Detailed analysis of specific outbreaks
- **Custom Reports:** User-defined report templates

Features:

- Auto-generated summaries
- Key metrics and visualizations
- Export formats:
 - PDF
 - DOCX (Word)
 - CSV (data tables)
- Scheduled report generation (architecture ready)

Access: `/api/v1/reports`

8. Monitoring & Evaluation

Purpose: Track data quality and system performance

Metrics:

- Reporting completeness (percentage of expected reports)
- Timeliness (days from case occurrence to reporting)
- Data quality indicators
- System performance metrics

Access: Integrated into dashboard views

User Roles and Permissions

The platform implements Role-Based Access Control (RBAC) with four user roles:

1. Public User (Role ID: 1)

Permissions:

- View aggregated global dashboard data
- View country-level public dashboards
- Read-only access to public reports

Restrictions:

- Cannot upload data
- Cannot view detailed case data
- Cannot access alerts
- Cannot generate forecasts

2. Country Data Officer (Role ID: 2)

Permissions:

- All Public User permissions
- Upload case data for assigned country
- View country-specific dashboard with detailed data
- View alerts for assigned country
- View forecasts for assigned country

Restrictions:

- Limited to assigned country only
- Cannot modify other countries' data
- Cannot access global administrative features

3. Epidemiologist (Role ID: 3)

Permissions:

- All Country Data Officer permissions
- Access to all countries' data
- Generate forecasts for any country
- View all alerts globally
- Access advanced analytics
- Generate custom reports
- View stratified data (age groups, gender, etc.)

Restrictions:

- Cannot manage users
- Cannot modify system settings

4. Admin (Role ID: 4)

Permissions:

- Full system access
 - All Epidemiologist permissions
 - User management (create, update, delete users)
 - Disease and country management
 - System configuration
 - Audit log access
 - Alert resolution and management
-

API Documentation

Base URL

- **Development:** <http://localhost:8000/api/v1>
- **Production:** <https://api.episphere.ai/api/v1>

Authentication

Most endpoints require JWT authentication. Include the token in the Authorization header:

```
Authorization: Bearer <access_token>
```

Get an access token by logging in:

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "user@example.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

API Endpoints

Authentication

Register User

```
POST /api/v1/auth/register
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "username": "username",
  "password": "password123",
  "full_name": "Full Name",
  "role_id": 2,
  "country_id": 1
}
```

Login

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded
```

```
username=user@example.com&password=password123
```

Get Current User

```
GET /api/v1/auth/me
Authorization: Bearer <token>
```

Cases

List Cases

```
GET /api/v1/cases?country_id=1&disease_id=1&start_date=2024-01-01&end_date=2024-12-31&skip=0&limit=100
Authorization: Bearer <token>
```

Query Parameters:

- `country_id` (optional): Filter by country
- `disease_id` (optional): Filter by disease
- `start_date` (optional): Start date (YYYY-MM-DD)
- `end_date` (optional): End date (YYYY-MM-DD)
- `skip` (optional): Pagination offset (default: 0)
- `limit` (optional): Results per page (default: 1000, max: 10000)

Create Case

```
POST /api/v1/cases
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "country_id": 1,
  "disease_id": 1,
  "date": "2024-01-15",
  "daily_cases": 25,
  "daily_deaths": 2,
  "cumulative_cases": 1500,
  "cumulative_deaths": 50
}
```

Upload Cases (CSV/Excel)

```
POST /api/v1/cases/upload
Authorization: Bearer <token>
Content-Type: multipart/form-data
```

```
file: <file>
country_id: 1
disease_id: 1
```

File Format: CSV or Excel with columns: date, daily_cases, daily_deaths, cumulative_cases, cumulative_deaths

Get Case Statistics

```
GET /api/v1/cases/stats?country_id=1&disease_id=1&start_date=2024-01-01&end_date=2024-12-31
Authorization: Bearer <token>
```

Alerts

List Alerts

```
GET /api/v1/alerts?
country_id=1&disease_id=1&severity=high&status=triggered&skip=0&limit=100
Authorization: Bearer <token>
```

Query Parameters:

- `country_id` (optional): Filter by country
- `disease_id` (optional): Filter by disease
- `severity` (optional): low, moderate, high
- `status` (optional): triggered, investigating, resolved, false_positive
- `skip, limit`: Pagination

Get Alert Details

```
GET /api/v1/alerts/{alert_id}
Authorization: Bearer <token>
```

Resolve Alert

```
POST /api/v1/alerts/{alert_id}/resolve
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "resolved",
  "notes": "Investigation completed, no outbreak confirmed"
}
```

Forecast

Generate Forecast

```
POST /api/v1/forecast/generate
Authorization: Bearer <token>
Content-Type: application/json

{
  "country_id": 1,
  "disease_id": 1,
  "horizon_days": 30,
  "model_type": "auto"
}
```

Parameters:

- `country_id` (required): Country to forecast
- `disease_id` (required): Disease to forecast
- `horizon_days` (optional): Forecast horizon in days (default: 30, max: 90)
- `model_type` (optional): "arima", "prophet", "lstm", or "auto" (default)

Response:

```
{
  "forecast_id": 123,
  "country_id": 1,
  "disease_id": 1,
  "model_used": "prophet",
  "forecast_data": [
    {
      "date": "2024-02-01",
      "predicted_cases": 45.2,
      "lower_bound": 38.1,
      "upper_bound": 52.3
    }
  ],
  "metrics": {
    "mae": 3.2,
    "rmse": 4.5,
    "mape": 8.5
  }
}
```

Get Forecast

```
GET /api/v1/forecast/{forecast_id}
Authorization: Bearer <token>
```

Dashboard**Global Dashboard Data**

```
GET /api/v1/dashboard/global?disease_id=1&start_date=2024-01-01&end_date=2024-12-31
Authorization: Bearer <token>
```

Response:

```
{
  "total_cases": 150000,
  "total_deaths": 5000,
  "countries": [
    {
      "country_id": 1,
      "country_name": "Country A",
      "cases": 50000,

```

```
    "deaths": 1500,  
    "incidence_per_100k": 250.5  
  },  
],  
"daily_trend": [  
  {  
    "date": "2024-01-01",  
    "cases": 1200,  
    "deaths": 45  
  }  
]  
}
```

Country Dashboard Data

```
GET /api/v1/dashboard/country/{country_id}?disease_id=1&start_date=2024-01-01&end_date=2024-12-31  
Authorization: Bearer <token>
```

Countries

List Countries

```
GET /api/v1/countries?region_id=1&skip=0&limit=100  
Authorization: Bearer <token>
```

Diseases

List Diseases

```
GET /api/v1/diseases?skip=0&limit=100  
Authorization: Bearer <token>
```

Reports

Generate Report

```
POST /api/v1/reports/generate  
Authorization: Bearer <token>  
Content-Type: application/json
```

```
{
```

```
"report_type": "weekly_bulletin",  
"country_id": 1,  
"disease_id": 1,  
"start_date": "2024-01-01",  
"end_date": "2024-01-07"  
}
```

List Reports

```
GET /api/v1/reports?country_id=1&report_type=weekly_bulletin&skip=0&limit=100  
Authorization: Bearer <token>
```

Download Report

```
GET /api/v1/reports/{report_id}/download?format=pdf  
Authorization: Bearer <token>
```

Users (Admin Only)

List Users

```
GET /api/v1/users?role_id=2&skip=0&limit=100  
Authorization: Bearer <token>
```

Create User

```
POST /api/v1/users  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "email": "user@example.com",  
  "username": "username",  
  "password": "password123",  
  "full_name": "Full Name",  
  "role_id": 2,  
  "country_id": 1  
}
```

Update User


```
PUT /api/v1/users/{user_id}
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "full_name": "Updated Name",
  "is_active": true
}
```

Delete User

```
DELETE /api/v1/users/{user_id}
Authorization: Bearer <token>
```

Interactive API Documentation

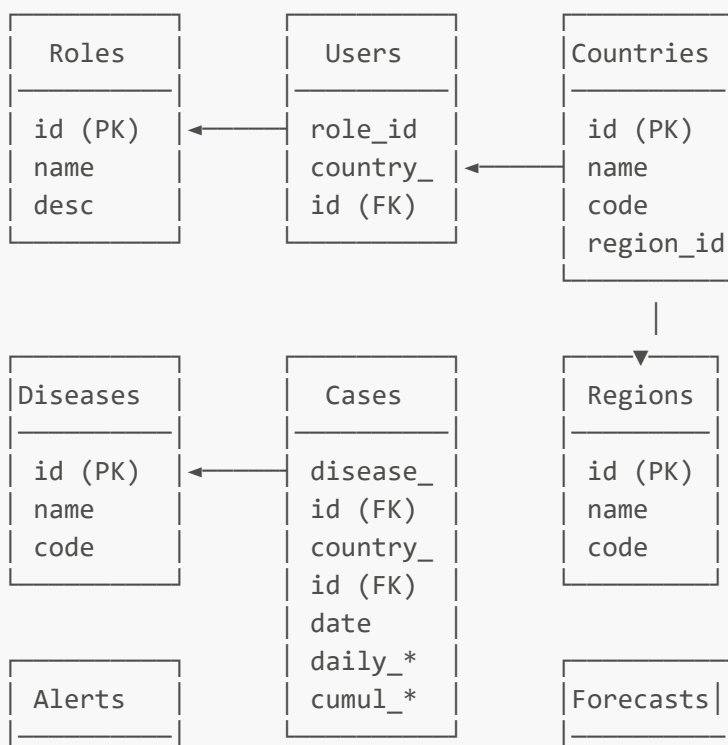
When the backend is running, visit:

- **Swagger UI:** <http://localhost:8000/docs>
- **ReDoc:** <http://localhost:8000/redoc>

These provide interactive documentation where you can test endpoints directly.

Database Schema

Entity Relationship Diagram



id (PK)	id (PK)
country_	country_
id (FK)	id (FK)
disease_	disease_
id (FK)	id (FK)
severity	model
status	forecast_
	data

Tables

roles

User roles for RBAC.

Column	Type	Description
id	INTEGER	Primary key
name	VARCHAR(50)	Role name (unique)
description	TEXT	Role description
created_at	TIMESTAMP	Creation timestamp

Default Roles:

- 1. public - Public user
- 2. country_data_officer - Country data officer
- 3. epidemiologist - Epidemiologist
- 4. admin - Administrator

users

User accounts.

Column	Type	Description
id	INTEGER	Primary key
email	VARCHAR(255)	Email address (unique)
username	VARCHAR(100)	Username (unique)
hashed_password	VARCHAR(255)	Bcrypt hashed password
full_name	VARCHAR(255)	Full name
role_id	INTEGER	Foreign key to roles
country_id	INTEGER	Foreign key to countries (nullable)
is_active	BOOLEAN	Account active status

Column	Type	Description
is_verified	BOOLEAN	Email verification status
created_at	TIMESTAMP	Account creation time
last_login	TIMESTAMP	Last login time

countries

Countries and territories.

Column	Type	Description
id	INTEGER	Primary key
name	VARCHAR(255)	Country name
code	VARCHAR(3)	ISO 3166-1 alpha-3 code
code_alpha2	VARCHAR(2)	ISO 3166-1 alpha-2 code
region_id	INTEGER	Foreign key to regions
population	INTEGER	Population count
latitude	FLOAT	Geographic latitude
longitude	FLOAT	Geographic longitude
created_at	TIMESTAMP	Creation timestamp

regions

Geographic regions (e.g., WHO regions).

Column	Type	Description
id	INTEGER	Primary key
name	VARCHAR(255)	Region name
code	VARCHAR(10)	Region code
description	TEXT	Region description

diseases

Disease definitions.

Column	Type	Description
id	INTEGER	Primary key
name	VARCHAR(255)	Disease name

Column	Type	Description
code	VARCHAR(50)	Disease code (e.g., ICD-10)
category	VARCHAR(100)	Disease category
description	TEXT	Disease description
is_active	BOOLEAN	Active status

cases

Time-series case data (TimescaleDB hypertable).

Column	Type	Description
id	INTEGER	Primary key
country_id	INTEGER	Foreign key to countries
disease_id	INTEGER	Foreign key to diseases
date	DATE	Case date
daily_cases	INTEGER	Daily new cases
daily_deaths	INTEGER	Daily new deaths
cumulative_cases	INTEGER	Cumulative cases
cumulative_deaths	INTEGER	Cumulative deaths
created_at	TIMESTAMP	Record creation time
updated_at	TIMESTAMP	Last update time

Indexes:

- Primary key on `id`
- Index on (`country_id`, `disease_id`, `date`) for fast lookups
- TimescaleDB hypertable on `date` for time-series optimization

alerts

Outbreak detection alerts.

Column	Type	Description
id	INTEGER	Primary key
country_id	INTEGER	Foreign key to countries
disease_id	INTEGER	Foreign key to diseases
severity	ENUM	low, moderate, high

Column	Type	Description
status	ENUM	triggered, investigating, resolved, false_positive
probability_score	FLOAT	Detection probability (0-1)
detection_method	VARCHAR(50)	ML method used
explanation	TEXT	Human-readable explanation
triggered_at	TIMESTAMP	Alert trigger time
resolved_at	TIMESTAMP	Resolution time (nullable)
resolved_by	INTEGER	Foreign key to users (nullable)
resolution_notes	TEXT	Resolution notes

forecasts

Forecast results.

Column	Type	Description
id	INTEGER	Primary key
country_id	INTEGER	Foreign key to countries
disease_id	INTEGER	Foreign key to diseases
model_used	VARCHAR(50)	Model name (arima, prophet, lstm)
forecast_data	JSON	Forecast values with dates
metrics	JSON	Model performance metrics
horizon_days	INTEGER	Forecast horizon
generated_at	TIMESTAMP	Generation time
generated_by	INTEGER	Foreign key to users

reports

Generated reports.

Column	Type	Description
id	INTEGER	Primary key
report_type	ENUM	weekly_bulletin, monthly_report, outbreak_report, custom
country_id	INTEGER	Foreign key to countries (nullable)
disease_id	INTEGER	Foreign key to diseases (nullable)
start_date	DATE	Report start date

Column	Type	Description
end_date	DATE	Report end date
file_path	VARCHAR(500)	Stored file path
format	VARCHAR(10)	pdf, docx, csv
generated_at	TIMESTAMP	Generation time
generated_by	INTEGER	Foreign key to users

audit_logs

Audit trail for system actions.

Column	Type	Description
id	INTEGER	Primary key
user_id	INTEGER	Foreign key to users
action	ENUM	create, update, delete, view, upload, download, login, logout
resource_type	VARCHAR(50)	Resource type (case, alert, etc.)
resource_id	INTEGER	Resource ID
details	JSON	Action details
ip_address	VARCHAR(45)	User IP address
created_at	TIMESTAMP	Action timestamp

Database Relationships

- **Users** → **Roles**: Many-to-one
- **Users** → **Countries**: Many-to-one (optional)
- **Cases** → **Countries**: Many-to-one
- **Cases** → **Diseases**: Many-to-one
- **Alerts** → **Countries**: Many-to-one
- **Alerts** → **Diseases**: Many-to-one
- **Forecasts** → **Countries**: Many-to-one
- **Forecasts** → **Diseases**: Many-to-one
- **Countries** → **Regions**: Many-to-one

Machine Learning Components

Outbreak Detection Engine

Location: backend/app/ml/outbreak_detection.py

Class: OutbreakDetectionEngine

Methods

`detect_outbreak(dates, daily_cases, country_name, disease_name)`

Main detection method that runs all algorithms and aggregates results.

Parameters:

- `dates`: List of date objects
- `daily_cases`: List of daily case counts
- `country_name`: Country name (for explanations)
- `disease_name`: Disease name (for explanations)

Returns:

```
{
  "alert_triggered": bool,
  "severity": "low" | "moderate" | "high" | None,
  "probability_score": float, # 0-1
  "detection_method": str,
  "explanation": str,
  "method_results": {
    "baseline": {...},
    "isolation_forest": {...},
    "cusum": {...},
    "seasonal": {...}
  }
}
```

Detection Algorithms

1. Baseline Statistical Threshold

Method: `_baseline_threshold_detection(cases_array)`

- Calculates mean and standard deviation of historical data
- Flags if current value exceeds `mean + 2*SD`
- Simple and interpretable

Use Case: Quick detection of sudden spikes

2. Isolation Forest

Method: `_isolation_forest_detection(cases_array)`

- Unsupervised anomaly detection
- Uses scikit-learn's IsolationForest
- Identifies outliers in case patterns
- Effective for non-linear anomalies

Use Case: Detecting unusual patterns that don't follow normal distribution

3. CUSUM Change Detection

Method: `_cusum_detection(cases_array)`

- Cumulative Sum control chart
- Detects sustained increases in cases
- Less sensitive to single-day spikes
- Good for gradual trend changes

Use Case: Detecting gradual increases that may indicate early outbreak

4. Seasonal Anomaly Detection

Method: `_seasonal_anomaly_detection(cases_array, dates)`

- Accounts for seasonal patterns
- Compares current values to expected seasonal baseline
- Uses moving average with seasonal adjustment

Use Case: Detecting outbreaks during expected low-season periods

Severity Classification

The engine aggregates results from all methods and assigns severity:

- **High:** Multiple methods agree, high probability (>0.7)
- **Moderate:** Some methods agree, medium probability ($0.4-0.7$)
- **Low:** Weak signal, low probability (<0.4)
- **None:** No significant signal detected

Forecasting Engine

Location: `backend/app/ml/forecasting.py`

Class: `ForecastingPipeline`

Methods

`generate_forecast(dates, values, horizon_days, model_type)`

Generates forecast using best available model.

Parameters:

- `dates`: Historical dates
- `values`: Historical daily case counts
- `horizon_days`: Number of days to forecast (default: 30, max: 90)
- `model_type`: "arima", "prophet", "lstm", or "auto" (default)

Returns:

```
{
  "forecast_data": [
    {
      "date": "2024-02-01",
      "predicted": float,
      "lower_bound": float,
      "upper_bound": float
    }
  ],
  "model_used": str,
  "metrics": {
    "mae": float, # Mean Absolute Error
    "rmse": float, # Root Mean Squared Error
    "mape": float # Mean Absolute Percentage Error
  }
}
```

Forecasting Models**1. ARIMA**

Method: `_forecast_arma(series, future_dates, horizon_days)`

- AutoRegressive Integrated Moving Average
- Statistical time-series model
- Automatically selects optimal parameters (p, d, q)
- Good for short-term forecasts with trends

Strengths:

- Fast training
- Interpretable
- Good for linear trends

Limitations:

- Assumes linear relationships
- May struggle with complex seasonality

2. Prophet

Method: `_forecast_prophet(series, future_dates, horizon_days)`

- Facebook's Prophet forecasting tool
- Handles seasonality, holidays, and changepoints
- Robust to missing data
- Excellent for seasonal patterns

Strengths:

- Handles multiple seasonalities
- Robust to outliers
- Good default parameters

Limitations:

- Slower than ARIMA
- Requires more data

3. LSTM

Method: `_forecast_lstm(series, future_dates, horizon_days)` (Placeholder)

- Long Short-Term Memory neural network
- Deep learning approach
- Can capture complex non-linear patterns

Status: Architecture ready, full implementation pending

Future Implementation:

- PyTorch-based LSTM model
- Sequence-to-sequence architecture
- Training on historical data

Model Selection

When `model_type="auto"`, the pipeline:

1. Trains all available models
2. Validates each on a hold-out set
3. Selects model with lowest validation error
4. Generates forecast with selected model

Validation Metrics:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- MAPE (Mean Absolute Percentage Error)

Frontend Structure

Next.js App Directory Structure

The frontend uses Next.js 14 with the App Router:

```
frontend/app/  
├─ layout.tsx      # Root layout with navigation
```

```
|— page.tsx          # Landing page
|— globals.css       # Global styles
|— auth/
|   |— login/
|   |   |— page.tsx  # Login page
|   |   |— register/
|   |       |— page.tsx # Registration page
|— dashboard/
|   |— global/
|   |   |— page.tsx  # Global surveillance dashboard
|   |   |— country/
|   |       |— [id]/
|   |           |— page.tsx # Country-specific dashboard
|— alerts/
|   |— page.tsx      # Alerts center
|— upload/
|   |— page.tsx      # Data upload page
```

Components

Navbar.tsx

Main navigation component with:

- Logo and branding
- Navigation links
- User authentication status
- Logout functionality

Dashboard Components

StatsCards.tsx

Displays key metrics as cards:

- Total cases
- Total deaths
- Incidence rate
- Growth rate
- Case fatality rate

TimeSeriesChart.tsx

Time-series visualization using ECharts:

- Daily cases line chart
- 7-day moving average
- Interactive tooltips
- Date range selection

GlobalMap.tsx

World map visualization:

- Country-level case distribution
- Color-coded by case count
- Interactive tooltips
- Zoom and pan controls

API Client

Location: `frontend/lib/api.ts`

Centralized API client with:

- Axios instance configuration
- Automatic token injection
- Error handling
- Request/response interceptors

Usage:

```
import api from '@lib/api';

// Get global dashboard data
const data = await api.get('/dashboard/global');

// Upload file
const formData = new FormData();
formData.append('file', file);
await api.post('/cases/upload', formData);
```

Styling

- **Framework:** TailwindCSS 3.3.6
- **Configuration:** `tailwind.config.js`
- **Global Styles:** `app/globals.css`
- **Responsive Design:** Mobile-first approach

State Management

Currently uses React hooks for local state. For complex state management, consider:

- Context API for global state
- React Query for server state
- Zustand or Redux for complex state

Setup and Installation

Prerequisites

- **Docker** and **Docker Compose** (recommended)
- OR **Python 3.10+** and **Node.js 18+** (manual setup)
- **PostgreSQL** (if not using Docker)

Option 1: Docker Setup (Recommended)

1. **Clone or navigate to the project directory**
2. **Start all services:**

```
docker-compose up -d
```

3. **Initialize the database:**

```
docker-compose exec backend python scripts/init_db.py
```

4. **Access the application:**

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:8000>
- API Docs: <http://localhost:8000/docs>

Option 2: Manual Setup

Backend Setup

1. **Navigate to backend directory:**

```
cd backend
```

2. **Create virtual environment:**

```
python -m venv venv  
# Windows  
venv\Scripts\activate  
# Linux/Mac  
source venv/bin/activate
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

4. **Configure environment:** Create `.env` file:

```
DATABASE_URL=sqlite:///./episphere.db
SECRET_KEY=your-secret-key-here
DEBUG=True
CORS_ORIGINS=http://localhost:3000
```

5. **Initialize database:**

```
python scripts/init_db.py
```

6. **Run migrations** (if using PostgreSQL):

```
alembic upgrade head
```

7. **Start server:**

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend Setup

1. **Navigate to frontend directory:**

```
cd frontend
```

2. **Install dependencies:**

```
npm install
```

3. **Configure environment:** Create `.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

4. **Start development server:**

```
npm run dev
```

Creating Initial User

After initialization, create a user via API:

```
curl -X POST "http://localhost:8000/api/v1/auth/register" \
-H "Content-Type: application/json" \
-d '{
  "email": "admin@episphere.ai",
  "username": "admin",
  "password": "admin123",
  "full_name": "Admin User",
  "role_id": 4
}'
```

Or use the Swagger UI at <http://localhost:8000/docs>

Configuration

Backend Configuration

File: `backend/app/core/config.py`

Environment Variables (set in `.env`):

Variable	Default	Description
DATABASE_URL	sqlite:///./episphere.db	Database connection string
REDIS_URL	redis://localhost:6379/0	Redis connection string
SECRET_KEY	change-this-secret-key-in-production	JWT secret key
ALGORITHM	HS256	JWT algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	30	Token expiration time
DEBUG	True	Debug mode
CORS_ORIGINS	http://localhost:3000	Allowed CORS origins (comma-separated)
UPLOAD_DIR	uploads	File upload directory
MAX_UPLOAD_SIZE	52428800	Max upload size in bytes (50MB)
FORECAST_HORIZON_DAYS	30	Default forecast horizon
OUTBREAK_DETECTION_WINDOW	14	Detection window in days

Frontend Configuration

File: frontend/.env.local

Variable	Description
NEXT_PUBLIC_API_URL	Backend API URL

Docker Configuration

File: docker-compose.yml

Services:

- **postgres:** PostgreSQL with TimescaleDB
- **redis:** Redis cache
- **backend:** FastAPI application
- **frontend:** Next.js application

Ports:

- 5432: PostgreSQL
- 6379: Redis
- 8000: Backend API
- 3000: Frontend

Security

Authentication

- **JWT Tokens:** Stateless authentication
- **Token Expiration:** 30 minutes (configurable)
- **Password Hashing:** bcrypt with salt rounds
- **Token Storage:** Frontend stores in memory/localStorage

Authorization

- **Role-Based Access Control (RBAC):** Four user roles
- **Endpoint Protection:** Decorators for role requirements
- **Data Filtering:** Users see only authorized data

Data Protection

- **Input Validation:** Pydantic schemas for all inputs
- **SQL Injection Prevention:** SQLAlchemy ORM (parameterized queries)
- **XSS Protection:** React's built-in escaping
- **CORS:** Configured allowed origins
- **Rate Limiting:** Architecture ready (to be implemented)

Audit Logging

All significant actions are logged:

- User logins/logouts
- Data uploads
- Case creation/updates
- Alert resolutions
- Report generation

Table: `audit_logs`

Best Practices

1. **Never commit secrets:** Use `.env` files (in `.gitignore`)
 2. **Use strong passwords:** Enforce password policies
 3. **Regular updates:** Keep dependencies updated
 4. **HTTPS in production:** Always use SSL/TLS
 5. **Monitor audit logs:** Regular review of system activity
-

Development Guide

Code Structure

Backend

- **API Endpoints:** `backend/app/api/v1/endpoints/`
- **Business Logic:** `backend/app/services/`
- **Database Models:** `backend/app/db/models.py`
- **Schemas:** `backend/app/schemas/`
- **ML Components:** `backend/app/ml/`

Frontend

- **Pages:** `frontend/app/`
- **Components:** `frontend/components/`
- **Utilities:** `frontend/lib/`

Adding a New Endpoint

1. **Create schema** in `backend/app/schemas/`
2. **Add service method** in `backend/app/services/`
3. **Create endpoint** in `backend/app/api/v1/endpoints/`
4. **Register route** in `backend/app/api/v1/api.py`

Adding a New ML Model

1. **Implement model class** in `backend/app/ml/`
2. **Add to detection/forecasting pipeline**
3. **Update service** to use new model
4. **Add tests** (when test suite is implemented)

Database Migrations

1. Create migration:

```
alembic revision --autogenerate -m "description"
```

2. Review migration in `backend/alembic/versions/`

3. Apply migration:

```
alembic upgrade head
```

Testing

Backend (when implemented):

```
cd backend  
pytest
```

Frontend (when implemented):

```
cd frontend  
npm test
```

Code Quality

- **Backend:** Follow PEP 8, use type hints, write docstrings
- **Frontend:** Follow ESLint rules, use TypeScript strictly
- **Both:** Write comments for complex logic

Deployment

Production Checklist

- ☐ Set `DEBUG=False` in environment
- ☐ Generate strong `SECRET_KEY`
- ☐ Configure production database (PostgreSQL)
- ☐ Set proper `CORS_ORIGINS`
- ☐ Configure HTTPS/SSL
- ☐ Set up email service (for notifications)
- ☐ Configure Redis for caching
- ☐ Set up monitoring (Prometheus/Grafana)

- ☐ Configure backup strategy
- ☐ Set up log aggregation
- ☐ Review security settings
- ☐ Run database migrations
- ☐ Build frontend: `npm run build`
- ☐ Test all endpoints
- ☐ Load testing

Docker Production Deployment

1. **Update** `docker-compose.yml` with production settings

2. **Build images:**

```
docker-compose build
```

3. **Start services:**

```
docker-compose up -d
```

4. **Run migrations:**

```
docker-compose exec backend alembic upgrade head
```

Environment-Specific Configuration

Create separate `.env` files:

- `.env.development`
- `.env.staging`
- `.env.production`

Load appropriate file based on `ENVIRONMENT` variable.

Scaling

Horizontal Scaling:

- Multiple backend instances behind load balancer
- Shared Redis for session/cache
- Shared PostgreSQL database

Database Scaling:

- Read replicas for read-heavy operations
- TimescaleDB for time-series optimization

- Connection pooling
-

Troubleshooting

Common Issues

Backend Won't Start

Port already in use:

```
# Windows
netstat -ano | findstr ":8000"
# Linux/Mac
lsof -i :8000
```

Database connection error:

- Check `DATABASE_URL` in `.env`
- Ensure PostgreSQL is running
- Verify credentials

Module not found:

- Activate virtual environment
- Run `pip install -r requirements.txt`

Frontend Won't Start

Port conflict:

- Change port: `npm run dev -- -p 3001`

API connection error:

- Check `NEXT_PUBLIC_API_URL` in `.env.local`
- Ensure backend is running
- Check CORS settings

Build errors:

- Delete `node_modules` and `.next`
- Run `npm install` again

Database Issues

Migration errors:

```
# Reset database (development only)
alembic downgrade base
alembic upgrade head
```

TimescaleDB not enabled:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
SELECT create_hypertable('cases', 'date', if_not_exists => TRUE);
```

ML Model Issues

Prophet installation:

- May require system dependencies
- On Linux: `sudo apt-get install build-essential`

PyTorch CUDA:

- For GPU support, install CUDA-enabled PyTorch
- Check: `python -c "import torch; print(torch.cuda.is_available())"`

Getting Help

1. Check logs:

```
docker-compose logs -f backend
docker-compose logs -f frontend
```

2. Review API documentation: <http://localhost:8000/docs>

3. Check database:

```
docker-compose exec postgres psql -U episphere -d episphere_db
```

4. Review error messages in browser console (F12)

Additional Resources

Documentation Files

- [README.md](#) - Main project readme
- [SETUP.md](#) - Detailed setup instructions
- [QUICK_START.md](#) - Quick start guide

- [START_SERVERS.md](#) - Server startup guide
- [PROJECT_SUMMARY.md](#) - Project completion summary

External Documentation

- [FastAPI Documentation](#)
- [Next.js Documentation](#)
- [SQLAlchemy Documentation](#)
- [TimescaleDB Documentation](#)
- [ECharts Documentation](#)

Support

For issues, questions, or contributions:

1. Review this documentation
2. Check existing issues
3. Contact the development team

Document Version: 1.0

Last Updated: 2024

Platform Version: 1.0.0